ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

КУРСОВА РАБОТА

Дисциплина: „Глобални Бази Данни“

Тема: Q&A Forum

Изготвил:

Борилсав Желев

Фак. № 381223038

Група: 92

III курс

Ръководител:

Доц. Д-р Петко Данов

София, 2026

1. Въведение

Q&A Forum е уеб приложение, в което потребителите могат да задават въпроси, да публикуват отговори и да гласуват за тях. Приложението е изградено по модела на популярни платформи като Stack Overflow.

Целта на проекта е да демонстрира работа с MongoDB чрез реални операции върху база данни — включително array полета, referencing между колекции и aggregation pipeline за статистически справки.

Приложението е изградено с React за потребителски интерфейс, Node.js и Express за backend и MongoDB за съхранение на данните. За контейнеризация е използван Docker с bridge мрежа, съвместима с Windows.

2. Кратка теория за MongoDB

MongoDB е NoSQL документна база данни, която съхранява данните като BSON документи (Binary JSON). Вместо таблици и редове, MongoDB работи с колекции от документи с гъвкава структура.

Три основни концепции, демонстрирани в проекта:

- Array fields — MongoDB позволява полетата да съдържат масиви от стойности или от референции към други документи. Например votes: [ObjectId] съхранява кои потребители са гласували.
- Referencing — документите в различни колекции могат да се свързват чрез ObjectId. Например authorId в Question реферира към документ от колекцията users.
- Aggregation pipeline — серия от трансформации ($match, $lookup, $addFields, $group, $sort) върху документи, позволяваща сложни справки и статистики.

MongoDB поддържа и индекси (включително multikey индекси върху array полета), които ускоряват търсенето по често използвани полета.

3. Структура на проекта

Проектът е организиран в три основни части: backend, frontend и конфигурация за Docker. Всяка от тях се изпълнява в отделен контейнер.

```
qa-forum/
├── docker-compose.yml       # оркестрация на 3 контейнера
├── backend/
│   ├── Dockerfile
│   ├── server.js            # Express сървър
│   ├── models/
│   │   ├── User.js          # Mongoose схема — users
│   │   ├── Question.js      # Array fields + Referencing
│   │   └── Answer.js        # Referencing + Array fields
│   └── routes/
│       ├── questions.js     # Aggregation pipeline
│       └── answers.js       # $addToSet, $pull
└── frontend/
    ├── Dockerfile
```

```
    ├── vite.config.js       # host: 0.0.0.0 (Windows Docker)
    └── src/
        ├── App.jsx
        └── api.js
```

Фиг. 1. Структура на проекта

Файлът docker-compose.yml дефинира три сервиза — mongo, backend и frontend — свързани в обща bridge мрежа qa_net. Тъй като Windows не поддържа host network mode в Docker, се използва bridge мрежата с вградена DNS резолюция по имена на сервизи.

---

4. База данни и колекции

Приложението използва база данни qa_forum с три колекции: users, questions и answers. Между тях съществуват референции (referencing), а в questions и answers се използват array полета.

4.1 Колекция users

```
// users collection
{
  _id:            ObjectId,
  name:           String,
  email:          String,   // unique
  avatarInitials: String,
  color:          String,
  createdAt:      Date
}
```

Фиг. 2. Схема на колекция users

4.2 Колекция questions

```
// questions collection
{
  _id:      ObjectId,
  authorId: ObjectId,    // REFERENCING → users._id
  title:    String,
  body:     String,
  tags:     [String],    // ARRAY FIELD — масив от тагове
  votes:    [ObjectId],  // ARRAY FIELD — масив от user refs
  views:    Number,
  createdAt: Date
}
```

Фиг. 3. Схема на колекция questions с array полета

Полетата tags и votes са array полета. Tags съхранява тагове като стрингове (например ["mongodb", "aggregation"]), а votes съдържа ObjectId-та на потребителите, гласували за въпроса. Върху tags е създаден multikey индекс за бързо търсене по таг.

4.3 Колекция answers

```
// answers collection
{
  _id:        ObjectId,
  questionId: ObjectId,  // REFERENCING → questions._id
  authorId:   ObjectId,  // REFERENCING → users._id
```

```
  body:       String,
  votes:      [ObjectId], // ARRAY FIELD — масив от user refs
  isAccepted: Boolean,
  createdAt:  Date
}
```

Фиг. 4. Схема на колекция answers с referencing

5. Основни функции

Приложението предоставя следните основни функционалности:

- Задаване на въпроси с заглавие, описание и тагове (array field)
- Публикуване на отговори към конкретен въпрос
- Гласуване за въпроси и отговори (toggle — добавяне/премахване от votes[])
- Приемане на отговор от автора на въпроса
- Филтриране по таг чрез индексирания tags[] масив
- Сортиране по брой гласове, нови или брой отговори
- Преглед на MongoDB схемите и реален aggregation pipeline резултат
- Автоматично seed на 4 потребители при първо стартиране

Всяка операция се изпълнява в реално време срещу MongoDB базата данни — няма in-memory данни или mock обекти. Всички промени се запазват в Docker volume mongo_data.

6. MongoDB операции в проекта

6.1 Добавяне на въпрос

За добавяне на нов документ се използва create() на Mongoose (обвива insertOne()). Таговете се нормализират до масив преди записването.

```
// routes/questions.js
const tagsArray = String(tags).split(',').map(t => t.trim().toLowerCase());

const question = await Question.create({
  title, body,
  tags: tagsArray,  // Array field
  authorId,         // Reference към users
});
```

Фиг. 5. Добавяне на въпрос с array field

6.2 Извличане с populate (Referencing)

При извличане на единичен въпрос се използва populate() за разрешаване на референцията authorId и замяна с реален обект от колекцията users.

```
// routes/questions.js
const question = await Question
  .findByIdAndUpdate(id, { $inc: { views: 1 } }, { new: true })
  .populate('authorId', 'name avatarInitials color');
  // authorId ObjectId → { name, avatarInitials, color }
```

Фиг. 6. Извличане с populate — разрешаване на референция

6.3 Гласуване — $addToSet и $pull върху Array

Гласуването добавя или премахва userId от масива votes[]. Използват се атомарните оператори $addToSet (добавя само ако не съществува) и $pull (премахва по стойност).

```
// routes/questions.js — toggle глас
const hasVoted = question.votes.some(v => v.equals(uid));

await Question.findByIdAndUpdate(id,
  hasVoted
    ? { $pull:     { votes: uid } }  // премахва от array
    : { $addToSet: { votes: uid } },  // добавя без дублиране
  { new: true }
);
```

Фиг. 7. Атомарна работа с array field — $addToSet / $pull

6.4 Приемане на отговор — updateMany + findByIdAndUpdate

При приемане на отговор първо се нулира isAccepted на всички отговори за въпроса, след което се маркира конкретният.

```
// routes/answers.js
// Нулиране на всички отговори за въпроса
await Answer.updateMany(
  { questionId: answer.questionId },
  { $set: { isAccepted: false } }
);

// Маркиране на приетия отговор
await Answer.findByIdAndUpdate(id,
  { $set: { isAccepted: true } },
  { new: true }
);
```

Фиг. 8. Приемане на отговор — updateMany + findByIdAndUpdate

7. Aggregation и статистика

Aggregation pipeline се използва в два основни случая: при зареждане на списъка с въпроси (сортирани по брой гласове/отговори) и при генериране на статистики.

7.1 Главен pipeline — GET /api/questions

Следният pipeline демонстрира $lookup (join с две колекции), $unwind, $addFields с $size за изчисляване на брой от array, $sort и $project:

```
db.questions.aggregate([
  { $match: matchStage },
  {
    $lookup: {                    // JOIN с answers
      from:         'answers',
      localField:   '_id',
      foreignField: 'questionId',
      as:           'answers'
    }
  },
  {
    $lookup: {                    // JOIN с users (REFERENCING)
      from:         'users',
```

```
      localField:   'authorId',
      foreignField: '_id',
      as:           'author'
    }
  },
  { $unwind: '$author' },
  {
    $addFields: {
      voteCount:   { $size: '$votes' },    // размер на array
      answerCount: { $size: '$answers' },  // размер на array
      hasAccepted: { $in: [true, '$answers.isAccepted'] }
    }
  },
  { $sort: { voteCount: -1 } },
  { $project: { title:1, tags:1, voteCount:1, 'author.name':1 } }
])
```

Фиг. 9. Aggregation pipeline — зареждане на въпроси

7.2 Статистически pipeline — GET /api/answers/stats

Вторият pipeline използва $group за групиране по въпрос и $avg/$sum за агрегирани стойности:

```
db.answers.aggregate([
  {
    $group: {
      _id:         '$questionId',
      answerCount: { $sum: 1 },
      avgVotes:    { $avg: { $size: '$votes' } },
      totalVotes:  { $sum: { $size: '$votes' } },
      hasAccepted: { $max: { $cond: ['$isAccepted', 1, 0] } }
    }
  },
  {
    $lookup: {    // JOIN с questions за заглавието
      from: 'questions', localField: '_id',
      foreignField: '_id', as: 'question'
    }
  },
  { $unwind: '$question' },
  { $sort: { answerCount: -1 } }
])
```

Фиг. 10. Статистически aggregation pipeline

8. Индекси и оптимизация

В моделите са дефинирани индекси за ускоряване на най-честите заявки:

```
// models/Question.js
questionSchema.index({ tags: 1 });
// Multikey индекс върху array — при $match: { tags: 'mongodb' }
// MongoDB автоматично индексира всеки елемент от масива

questionSchema.index({ title: 'text', body: 'text' });
// Text индекс за full-text търсене

// models/Answer.js
answerSchema.index({ questionId: 1, createdAt: -1 });
// Compound индекс — бързо намиране на отговорите на въпрос,
// сортирани по дата
```

Фиг. 11. Индекси в Mongoose моделите

Multikey индексът върху tags[] е особено важен — MongoDB автоматично индексира всеки елемент от масива поотделно. Това позволява бързо филтриране по таг без collection scan.

Compound индексът { questionId: 1, createdAt: -1 } в answers оптимизира конкретната заявка за извличане на отговорите на даден въпрос, сортирани по дата.

---

9. Docker конфигурация (Windows)

Приложението е напълно контейнеризирано с Docker Compose. Тъй като Windows не поддържа host network mode, е използвана bridge мрежа с вградена DNS резолюция.

9.1 docker-compose.yml

```
services:
  mongo:
    image: mongo:7
    volumes:
      - mongo_data:/data/db   # персистентност на данните
    healthcheck:
      test: ['CMD', 'mongosh', '--eval', "db.adminCommand('ping')"]

  backend:
    build: ./backend
    environment:
      - MONGO_URI=mongodb://mongo:27017/qa_forum
      # 'mongo' = DNS име на сервиза в bridge мрежата
    depends_on:
      mongo:
        condition: service_healthy

  frontend:
    build: ./frontend
    ports: ['5173:5173']

networks:
  qa_net:
    driver: bridge
```

Фиг. 12. docker-compose.yml — bridge мрежа за Windows

9.2 Vite конфигурация за Windows Docker

Критична настройка за Windows — Vite трябва да слуша на 0.0.0.0, а не само на localhost, за да е достъпен от хоста. Proxy-то препраща /api заявките към backend сервиза по DNS:

```
// frontend/vite.config.js
export default defineConfig({
  server: {
    host: '0.0.0.0',   // ЗАДЪЛЖИТЕЛНО за Windows Docker
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://backend:5000',  // Docker DNS
        changeOrigin: true,
      },
```

```
      },
    },
  });
```

Фиг. 13. Vite с host 0.0.0.0 и Docker DNS proxy

10. Как работи приложението

Потребителят избира своя профил от падащото меню в горната лента (реален user документ от MongoDB). При задаване на въпрос, React изпраща POST заявка към /api/questions.

Vite проксира заявката от localhost:5173 към backend:5000 чрез Docker bridge мрежата. Express валидира данните и извиква Question.create() в MongoDB. Тагите се записват като array field.

При зареждане на списъка с въпроси backend изпълнява aggregation pipeline с два $lookup (към users и answers), изчислява voteCount чрез $size на votes[], и сортира резултата.

Гласуването изпраща POST към /api/questions/:id/vote. Backend проверява дали userId вече е в масива votes[] и съответно изпълнява $addToSet или $pull за атомарна промяна на масива.

Данните се запазват в Docker volume mongo_data и не се губят при рестартиране на контейнерите. За достъп до MongoDB Compass — порт 27017 е пробит към хоста.

11. Заключение

Проектът Q&A Forum демонстрира реална работа с MongoDB в контекста на уеб приложение с три взаимосвързани колекции.

Използваните MongoDB концепции включват:

- Array fields — tags[] (стрингове) и votes[] (ObjectId референции) в questions и answers
- Referencing — authorId и questionId като ObjectId връзки между колекции, разрешавани с populate()
- Aggregation pipeline — $lookup, $unwind, $addFields, $size, $group, $avg, $sort, $project
- Атомарни array операции — $addToSet и $pull за toggle гласуване
- Multikey и compound индекси за оптимизация на заявките

Приложението е изцяло контейнеризирано с Docker Compose и е съвместимо с Windows чрез bridge мрежа и настройка на Vite host. При първо стартиране автоматично се инициализират seed данни.

Кодът е структуриран в отделни Mongoose модели с коментари, обясняващи всяка MongoDB концепция, и Express routes с подробни aggregation примери.